

Artemis AI Developer Starter Guide

Set up, run, train, ship. Everything a new developer needs on day one.

Artemis AI · October 3, 2026 · For developers, safe to share

Contents

1. Developer starter guide
2. Architecture: the application
3. Neural core: the model

Artemis AI: developer starter guide

Welcome. This guide gets you from a fresh checkout to running tests, training a small model, running the API, and shipping a change. Read the ground rules first; they are not optional.

1. What you are working on

Artemis AI builds its own AI, trained from scratch. Two codebases:

Codebase	What it is	Hosted on
`artemis-ai` (Python)	The model (Prime Core + ten neural experts), training, evaluation, inference, the orchestrator, chat tools, memory, plans and limits, and the API server	Azure Container Apps (`artemis-api`)
Website (static HTML/CSS/JS in `src/`)	Company site and the customer chat UI; talks to the API	Vercel (project `cutline-industries`), domain cutline-industries.studio

Read next: `docs/NEURAL_CORE.md` (the model) and `docs/ARCHITECTURE.md` (the application around it).

2. Ground rules

1. **Only Artemis's own model answers customers.** No OpenAI, Claude, Grok, Gemini or any outside model in the product or in Artemis's decisions (Blueprint Decisions 5 and 12). Search engines and code sandboxes are tools, not models.
2. **No secrets in code, chat, tickets or logs.** Secrets live in Azure Key Vault (`artemis-kv-6b22c1`) and reach the app as Key Vault references. Never paste a key or password anywhere to "share" it.
3. **Spending:** the system has a hard cap of $2,000,000 per month. Never start GPU VMs (about $98/hour each) without the owner's go-ahead.
4. **Tools are the application's job.** Permissions, limits, tool execution and deployment controls belong in `business.py`, `tools.py`, `server.py` and `infra/`, never inside the model.
5. **Report results honestly.** A claim that something trained, passed or improved needs the log or eval file behind it.
6. **Tests pass before you ship.** `python -m pytest tests` must be green.

3. Set up (10 minutes)

Requirements: Python 3.11+, git, and for the website tests Node 18+ with Playwright. A GPU is not needed for anything below.

```
cd artemis-ai
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt          # torch, numpy, pyyaml, tokenizers, safetensors,
pytest, psycopg
pip install transformers                 # optional: enables the export-equivalence tests
python -m pytest tests                   # about 45 s on a laptop CPU; expect all tests to
pass
```

4. Repository map

Path	What lives there
`artemis/model.py`	Prime Core transformer, `ExpertMixture` (router + ten experts), losses, generation
`artemis/train.py`	Training loop: resume, `--stop-at`, optimizer audit, routing diagnostics
`artemis/checkpoint.py`	Checkpoint format v2, strict loading, v1 → v2 migration, `inspect`
`artemis/evaluate.py`	Validation loss, per-expert routing, expert ablation, dense-baseline comparison
`artemis/data.py`, `tokenizer.py`, `corpus.py`, `pipeline.py`	Licensed data → tokenizer → token shards with domain labels and provenance
`artemis/infer.py`	Load a checkpoint and stream text, with expert routing traces
`artemis/specialize.py`, `knowledge.py`, `build_brains.py`	Fine-tuning (full or LoRA) and the self-knowledge / tool-use training sets
`artemis/export.py`	Export to Llama (dense) or Qwen2-MoE (experts) layout for vLLM
`artemis/orchestrator.py`	Plans each request, runs the tool loop, memory, audit, streaming events
`artemis/tools.py`	`web_search`, `web_fetch`, `run_python`
`artemis/memory.py`	Per-account notes (external memory, not weights)
`artemis/business.py`, `store.py`	Plans, limits, API keys, metering, waitlist; SQLite locally, PostgreSQL in Azure

Path	What lives there
`artemis/server.py`, `backends.py`	HTTP API; where answers come from (local checkpoint, vLLM server, or "still training")
`artemis/engine.py`	Decision engine: next experiment (UCB1), compute ramp, spending cap
`configs/models.yaml`	Model sizes (`smoke`, `proto`, `proto-dense`, `100m`, `1b`, `3b`, `8b`)
`configs/business.yaml`	Plans, prices, limits, which tools each plan gets
`brains/*.yaml`	Artemis and the ten planet personas
`infra/azure/`, `infra/local/`	Deploy, provisioning, GPU and prototype scripts
`tests/`	Everything above, tested on CPU

5. Quick starts

Train and evaluate the CPU prototype (about 90 minutes on 4 cores)

```
infra/local/run-prototype.sh           # corpus → tokenizer → shards → 3 runs → evaluations
cat runs/proto/eval.json               # per-expert routing, ablation, dense baseline
```

Talk to a checkpoint from the command line

```
python -m artemis.infer --checkpoint runs/proto/latest.pt --tokenizer runs/tok-proto.json \
    --prompt "Security review of billing:" --raw --trace
```

`--trace` reports which experts the router picked for the generated tokens.

Run the API locally on a checkpoint

```
ARTEMIS_CHECKPOINT=runs/proto/latest.pt ARTEMIS_TOKENIZER=runs/tok-proto.json
ARTEMIS_DB=dev.db PORT=8099 \
    python -m artemis.server
curl localhost:8099/v1/status
curl -N -X POST localhost:8099/v1/chat/stream -H "Content-Type: application/json" \
    -d '{"message": "Deploy checkout to eastus", "tier": "gpt-1-base"}'
```

`/v1/status` shows the loaded checkpoint, its step, and the experts. The stream returns `plan`, then `token` events, then `done`. (The prototype was never trained on chat, so its replies are not meaningful.) Add `ARTEMIS_LOCAL_CODE=1` to let `run_python` use a local subprocess; that is

for development only and is not a sandbox.

Run the website locally

```
npx serve src                          # from the website folder; open the printed URL, add
#chat for the chat
```

The API address is the `API` constant near the top of `src/polish.js`. To use a local API, change it and add your local origin to `ARTEMIS_ALLOWED_ORIGINS` on the API.

6. The API

Base URL: `https://artemis-api.whitemeadow-751c0637.eastus.azurecontainerapps.io`

Method and path	Auth	What it does
`GET /v1/status`	none	Whether a model is serving, which tools are configured, model info
`GET /v1/plans`, `GET /v1/brains`	none	Plans and prices; the personas
`POST /v1/chat`	optional `X-Artemis-Account`	One answer as JSON
`POST /v1/chat/stream`	optional `X-Artemis-Account`	Server-Sent Events: `plan`, `tool_call`, `tool_result`, `token`, `replace`, then `done` or `training`
`POST /v1/chat/completions`	`Authorization: Bearer art-...`	OpenAI-compatible request and response, billed per token
`GET/POST /v1/memory`, `DELETE /v1/memory/<id>`	API key	The account's memory notes
`POST /v1/waitlist`	none	`{"email": ...}`

Errors: 400 bad input, 401 missing or unknown key, 402 the plan doesn't include it, 429 limit reached. The `X-Artemis-Account` header is not proof of identity, so memory is only available with an API key.

Stream example:

```
event: tool_call
data: {"type": "tool_call", "name": "web_search", "arguments": {"query": "latest python
release"}}

event: tool_result
data: {"type": "tool_result", "name": "web_search", "ok": true, "summary": "Searched the web
for ...", "sources": [...]}

event: token
data: {"type": "token", "text": "Python 3.14 is "}

event: done
data: {"type": "done", "answer": "...", "tools_used": ["web_search"], "plan": "free", "tier":
"gpt-2-sft", ...}
```

7. The model in one page

- **Prime Core**: decoder-only transformer with RMSNorm, rotary embeddings, grouped-query attention, SwiGLU.
- **Experts**: at the layers in `moe_layers`, a learned router picks the top 2 of 10 SwiGLU experts per token and renormalizes their weights; their outputs add into the residual stream next to the core's own FFN.
- **Training loss** = next-token loss + 0.01 × load balance + 0.001 × router z-loss + `--route-coef` × routing supervision on domain-labeled tokens.
- **Checkpoints**: format 2, strict loading. Old dense checkpoints must be migrated (`python -m artemis.checkpoint migrate --src old.pt --dst new.pt --moe-every 2`); the original is never modified.
- **Evaluate** every run: `python -m artemis.evaluate --checkpoint ... --data ... --out eval.json [--baseline dense.pt]`. An expert counts as specialized only if removing it hurts its own domain clearly more than the others.

Train on GPUs (owner approval first):

```
infra/azure/provision-gpu.sh            # one ND H100 v5 VM (8 GPUs)
SIZE=100m STEPS=20000 infra/azure/train-on-vm.sh
python -m artemis.export --checkpoint runs/100m/latest.pt --tokenizer runs/tok.json --out
serve/artemis
infra/azure/serve-on-vm.sh              # vLLM; then set ARTEMIS_INFERENCE_URL on the API
infra/azure/stop-gpu.sh                 # always stop the VM when done
```

8. Chat tools

The model asks for a tool by writing `<tool_call>{"name": ..., "arguments": {...}}</tool_call>`. The orchestrator checks the plan (`business.authorize_tool`), runs it (`tools.Toolbox.execute`), and feeds the result back in a `<tool_result>` block labeled as untrusted data. Up to 4 calls per answer. Customers see each step in the chat.

To add a tool: 1. Add its spec to `TOOL_SPECS` and a branch in `Toolbox.execute` (`artemis/tools.py`). Validate every argument; return `ToolResult`. 2. Add its name to `ALL_TOOLS` (`artemis/business.py`) and to the plans that get it (`configs/business.yaml`). 3. Add training examples to `tools_set()` (`artemis/knowledge.py`), using real outputs. 4. Add tests next to `tests/test_tools.py`, including the failure and abuse cases. 5. If it reaches the network or runs code, document its safety limits in `docs/ARCHITECTURE.md`.

9. Configuration (API environment variables)

Variable	Meaning
`ARTEMIS_DATABASE_URL`	PostgreSQL URL (Key Vault reference in Azure). Otherwise `ARTEMIS_DB` (SQLite path, default `artemis.db`)
`ARTEMIS_INFERENCE_URL`, `ARTEMIS_INFERENCE_KEY`	Artemis's own vLLM server
`ARTEMIS_CHECKPOINT`, `ARTEMIS_TOKENIZER`	Load a checkpoint into the API process instead (needs torch)
`ARTEMIS_ALLOWED_ORIGINS`	Comma-separated website origins allowed to call the API
`ARTEMIS_TRUST_PROXY=1`	Use the client IP from Azure's ingress (needed for fair per-visitor limits)
`ARTEMIS_CODE_SESSIONS_ENDPOINT`	Azure dynamic sessions pool for `run_python`
`ARTEMIS_BRAVE_API_KEY`	Full web search (otherwise Wikipedia); store in Key Vault
`ARTEMIS_LOCAL_CODE=1`	Development only: run code in a local subprocess
`ARTEMIS_MODEL_ROUTER=1`	Let the trained orchestrator choose personas instead of keywords

10. Shipping a change

API (from `artemis-ai/`):

```
python -m pytest tests
az acr build -r artemisapi6b22c1 -t artemis-api:vN -f infra/azure/Dockerfile.api .
az containerapp update -g artm-rg -n artemis-api --image artemisapi6b22c1.azurecr.io/artemis-api:vN
curl https://<api>/v1/status
```

If a module is new, add it to the `COPY` line in `infra/azure/Dockerfile.api`; the API image has no torch.

Website: deploy `src/` to the Vercel project `cutline-industries` (output directory `src`, no build step), then check that the live files match what you deployed and test the chat on a phone width and a desktop width.

11. Azure resources (resource group `artm-rg`)

Resource	Name	Region	Purpose
Container App + environment	`artemis-api`, `artemis-api-env`	East US	The API
Container registry	`artemisapi6b22c1`	East US	API images
PostgreSQL flexible server	`artemis-db-6b22c1`, database `artemis`	East US 2	Accounts, usage, waitlist, memory
Key Vault (RBAC)	`artemis-kv-6b22c1`	East US	Database URL and other secrets
Dynamic session pool	`artemis-code`	East US	Isolated Python sandboxes for `run_python` (no internet)

New PostgreSQL servers are blocked in East US on this subscription, so the database is in East US 2.

12. Where things stand

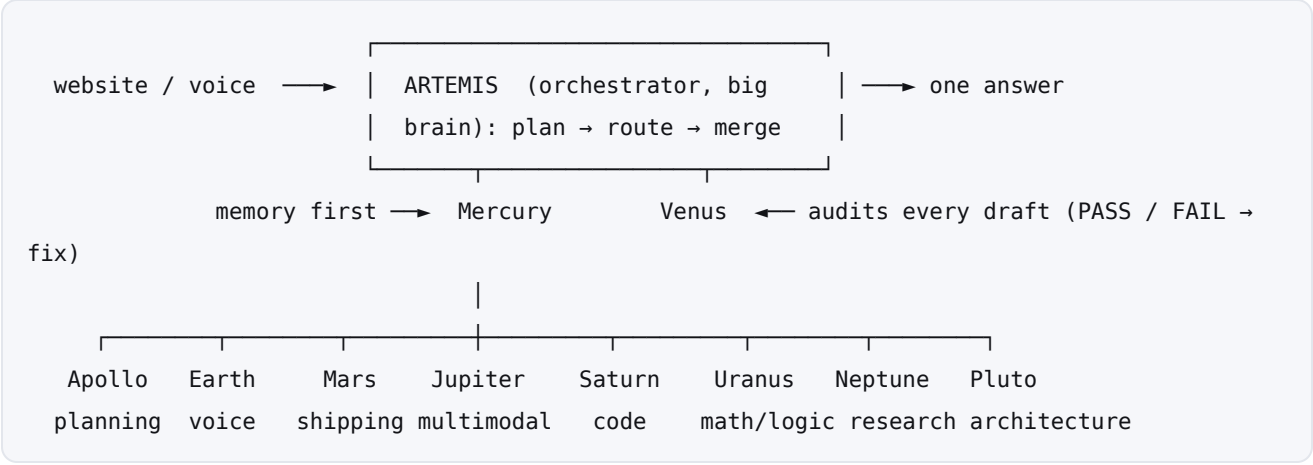
Built and tested on CPU: the neural core and its evaluations, checkpoints and migration, inference, export, orchestrator, chat tools, memory, plans and limits, streaming API, website chat. Running in Azure: API, database, Key Vault, code sandbox, website. Not done yet: any GPU training (H100 quota is 0), a production tokenizer and licensed training data, vLLM serving of the expert model. Until a model passes evaluation, chat answers "still training".

Artemis AI architecture

Artemis AI runs on one model it trains itself (Decisions 3, 5, 12). No outside AI model answers customers or makes decisions.

Inside the model (NEURAL_CORE.md): the Prime Core, a shared transformer, plus ten trainable neural experts (intent, architecture, planning, build, test, deploy, memory, security, observability, cost) that a learned router picks per token, top two at a time.

Around the model (this page): the application. Artemis the orchestrator and the ten planet brains are product personas (system prompts, optionally LoRA adapters on the Prime Core) that the website and plans expose. They are not the neural experts.



The eleven brains

Brain	Role	Used for
Artemis	Orchestrator, the big brain	Plans each request, picks brains, merges answers
Apollo	Navigator	Roadmaps, planning, strategy, business
Mercury	Memory	Persona for memory questions; retrieval itself comes from the memory store (`artemis/memory.py`)
Venus	Auditor	Checks every draft for errors and safety before it is sent
Earth	Voice	Spoken answers for the voice box, conversation, tone, translation
Mars	Shipping	Release readiness, go/no-go, rollback plans
Jupiter	Multimodal	Images, screenshots, charts, documents
Saturn	Code	Writing, fixing and testing code
Uranus	Math and logic	Calculation, statistics, step-by-step reasoning

Brain	Role	Used for
Neptune	Deep research	Cited research and source comparison
Pluto	Systems architecture	Infrastructure, scaling, cloud design

Definitions live in `brains/*.yaml` (role, instructions, rules, routing keywords, training focus, evaluations).

Chat tools

Customers' chat can search the web, read pages, and run Python, all on Artemis's own model: the model asks for a tool in its reply, and the application (`artemis/tools.py`, `artemis/orchestrator.py`) checks the plan, runs the tool, and hands the result back to the model marked as data, never instructions. Up to 4 tool calls per answer.

Tool	Runs on	Safety
`web_search`	Wikipedia search (no key), or Brave Search when `ARTEMIS_BRAVE_API_KEY` is set	Results are labeled untrusted
`web_fetch`	The API server	Public http(s) only; private, loopback, link-local and metadata addresses refused, redirects re-checked, 2 MB cap
`run_python`	Azure Container Apps dynamic sessions (`artemis-code` pool)	Hyper-V isolated sandbox per customer, no internet access, 60 s per run

Plan	Tools	Tool calls per day
Free	web search, web pages	20
Plus	all, including running code	300
Pro / Enterprise	all	unlimited
Team	all	1,000

The website shows each step ("Searching the web for …", sources, "Ran Python" with the code and output) and renders code blocks with a Copy button. Tool use is in the training data (`artemis/knowledge.py`, `tools` set; code outputs are real), but the model only uses tools well once it is trained; until then chat answers "still training".

Product tiers (the ladder)

Tier	Brains used	Memory	Venus audit
GPT-1 Base	Artemis alone	no	no

Tier	Brains used	Memory	Venus audit
GPT-2 SFT	best 1 specialist	no	no
GPT-3 Aligned RAG	up to 3 specialists	yes	yes
GPT-4 ASI Orchestrator	up to all 10	yes	yes

Customers can also open any single planet brain directly (for example "Saturn" for code).

How the weights are built

1. **Foundation** (`artemis/train.py`): the Prime Core and its ten experts, trained together from random initialization on licensed data (`artemis/data.py` refuses sources without a recorded license). Sizes in `configs/models.yaml`: 100M → 1B → 3B → 8B (experts at every second layer). See NEURAL_CORE.md.
2. **Specialists** (`artemis/specialize.py`): each planet is a LoRA adapter (a small set of extra weights) trained on that brain's conversations on top of the shared foundation. One server holds all ten.
3. **Orchestrator**: Artemis itself is fine-tuned on routing and merging examples collected from real traces.
4. **Serving** (`artemis/export.py`, `infra/azure/serve-on-vm.sh`): weights are exported to the Qwen2-MoE layout (Llama for dense models) and served by vLLM; the export is tested to give identical outputs. Without a GPU server, `ARTEMIS_CHECKPOINT` loads a checkpoint straight into the API process (`artemis/infer.py`).

Memory is not in the weights: per-account notes live in the database (`artemis/memory.py`), are retrieved by relevance for tiers with memory, and are placed in the prompt in a block labeled as retrieved memory. Only API-key accounts have memory. The website chat streams answers over `POST /v1/chat/stream`.

Business model (same structure as ChatGPT, Claude and Grok)

Defined once in `configs/business.yaml`, enforced by `artemis/business.py` and `artemis/server.py`:

Plan	Price	Ladder tier	Brains	Messages
Free	$0	GPT-2 SFT	Earth, Neptune, Saturn	30 / day
Plus	$20 / month	GPT-3 Aligned RAG	all ten	500 / day
Pro	$200 / month	GPT-4 ASI Orchestrator	all ten	unlimited

Plan	Price	Ladder tier	Brains	Messages
Team	$30 / seat / month	GPT-3 Aligned RAG	all ten	1,000 / day, admin console
Enterprise	contact sales	GPT-4 ASI Orchestrator	all ten	unlimited, SSO, API

Developer API: `POST /v1/chat/completions` (same request shape as OpenAI's), API keys (`art-...`, stored hashed), billed per million tokens per model (`artemis-base`, `artemis-sft`, `artemis-aligned`, `artemis-orchestrator`), 60 requests/minute, $5 starting credit. Over-limit requests get HTTP 402 (upgrade) or 429 (limit reached).

Self-knowledge in the weights

`artemis/knowledge.py` turns `brains/*.yaml` and `configs/business.yaml` into training conversations (who Artemis is, the eleven brains, plans, prices, API, policies, routing). Each brain's `training_sets` says which sets its weights learn:

Weights	Learns
Artemis (full fine-tune)	identity, business, brains, routing
Each planet (adapter)	identity, business, brains

`python -m artemis.build_brains ...` regenerates the data and trains all eleven. Change a price in `business.yaml`, rebuild, and the weights match.

How the system improves itself

`artemis/engine.py` is the decision engine ("own brain", stage 1): - picks the next experiment with UCB1 (mostly build on what scored best, sometimes explore), - unlocks larger model sizes only after the smaller size has a stable result, - ramps compute 1 → 4 → 16 → 27 VMs, advancing only after a stable run that improved, - refuses any run that would exceed the $2,000,000 monthly cap, - promotes a new model only if it beats the current one on held-out data by a margin.

In stage 2, once Artemis's own model passes its routing evaluation, it proposes experiments and routing plans itself (`ARTEMIS_MODEL_ROUTER=1`); the engine stays as the referee.

Running it

Step	Command
Tests (CPU)	`python -m pytest tests`

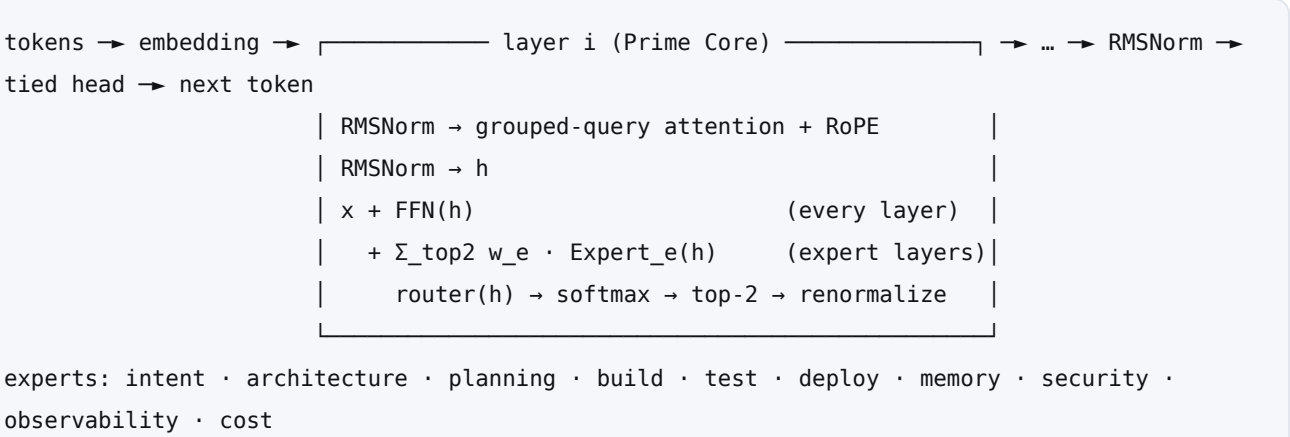
Step	Command
CPU prototype (neural core vs dense baseline)	`infra/local/run-prototype.sh`
Evaluate experts	`python -m artemis.evaluate --checkpoint ... --data ... --out eval.json`
Inspect / migrate a checkpoint	`python -m artemis.checkpoint inspect ...` / `migrate ...`
Run a checkpoint locally	`python -m artemis.infer --checkpoint ... --tokenizer ... --prompt "..." --trace`
Tokenizer	`python -m artemis.pipeline tokenizer --files corpus/*.txt --out runs/tok.json`
Data shards	`python -m artemis.pipeline prepare --sources sources.yaml --tokenizer runs/tok.json --out runs/data`
Start GPU VM	`infra/azure/provision-gpu.sh` (needs H100 quota; ~$98/hour)
Pretrain	`infra/azure/train-on-vm.sh`
Specialize a brain	`python -m artemis.specialize --brain saturn ...`
Export + serve	`python -m artemis.export ...`, then `infra/azure/serve-on-vm.sh`
Website API	`infra/azure/deploy-api.sh`, then set `ARTEMIS_INFERENCE_URL`
Stop GPU VM	`infra/azure/stop-gpu.sh`

Status

Built and tested on CPU: the neural core (Prime Core + ten experts, router, losses, diagnostics, checkpoints and migration, evaluations, CPU prototype against a dense baseline), brain definitions, specialization, export, orchestrator, memory store, streaming website API, business engine, self-knowledge training, decision engine. Not yet run: anything on GPUs. Blocked on Azure H100 quota.

Artemis neural core

One trainable **Prime Core** and **ten trainable neural experts**, inside one model (`artemis/model.py`). This is the model used by training (`artemis/train.py`), evaluation (`artemis/evaluate.py`), fine-tuning (`artemis/specialize.py`), export for serving (`artemis/export.py`), local inference (`artemis/infer.py`) and the API server (`artemis/server.py` → `backends.LocalBackend`).



Piece	Where	What it is
Prime Core	`ArtemisLM` (embedding, `Attention`, `FeedForward`, `RMSNorm`, head)	Shared decoder-only transformer. Runs on every token and generates every token.
Router	`ExpertMixture.router`	`Linear(d_model → 10)` per expert layer, learned. Softmax, keep top 2, renormalize the two weights to sum to 1.
Experts	`ExpertMixture.experts[0..9]`	Ten SwiGLU feed-forward modules, each with its own weights. Their weighted outputs are added to the Prime Core's residual stream.
Expert layers	`ModelConfig.moe_layers` / `moe_every`	Selected layers only (prototype: 1, 3, 5 of 6; GPU sizes: every second layer).
Load balancing	`balance_coef` × Switch-style loss	`10 · Σ_e (share of routing slots_e × mean router probability_e)`; 1.0 = perfectly even.
Router z-loss	`router_z_coef` × mean(logsumexp(logits)²)	Keeps router logits from growing without bound.
Routing supervision	`--route-coef` × NLL of the labeled domain's expert	Only on tokens whose domain is labeled. Teaches the router which expert a domain belongs to; at inference no labels exist and the router decides alone.

Piece	Where	What it is
Diagnostics	`metrics.jsonl`	Per step: each loss term, per-layer expert load, router entropy; at eval steps: router and per-expert gradient norms, validation loss.

Attention, normalization and positional encoding: RMSNorm (pre-norm), rotary embeddings, grouped-query attention (`n_kv_heads < n_heads`), SwiGLU in the Prime Core FFN and in every expert.

What is separate from the weights

Lives in the application, not the model	File
External persistent memory (per-account notes, BM25 retrieval, inserted into the prompt in a block labeled as retrieved memory)	`artemis/memory.py`, `artemis/orchestrator.py`
Permissions (API keys, plans, which tiers and brains an account may use)	`artemis/business.py`, `artemis/server.py`
Resource limits (messages per day, API rate and credit, memory size, history length, message length)	`artemis/business.py`, `artemis/memory.py`, `artemis/server.py`
Tool execution (web search, web pages, sandboxed Python), with per-plan permission and daily caps	`artemis/tools.py`, `artemis/orchestrator.py`, `artemis/business.py`
Deployment controls (GPU start/stop, $2M monthly cap, promotion gate)	`infra/azure/*`, `artemis/engine.py`
Planet brains (Apollo … Pluto): product personas = a system prompt and optionally a LoRA adapter on the Prime Core	`brains/*.yaml`, `artemis/orchestrator.py`, `artemis/specialize.py`

The **memory expert** is a neural module that learned to process memory-related text; it stores nothing between conversations. The **memory store** is a database table; it is never trained into the weights.

Conflict with earlier instructions

Earlier, the "ten brains" were the planet brains: prompts plus LoRA adapters, chosen by a keyword router. Those are role labels around one model, not neural experts. The current specification governs the neural architecture: the ten experts are trainable modules inside the model, chosen per token by a learned router. The planet brains remain as product personas (the website and plans reference them), are documented as such, and are not counted as experts. LoRA adapters attach only to the Prime Core, never to experts or routers.

Checkpoints

Format 2 (`artemis/checkpoint.py`): `format_version`, `arch: artemis-prime-moe`, config, all tensors, optimizer, step, RNG state, and a `params` manifest of every tensor's shape. Loading is strict: a missing or unexpected tensor raises an error naming it.

Format 1 (dense, before the neural core) is refused with instructions to migrate: `python -m artemis.checkpoint migrate --src old.pt --dst new.pt --moe-every 2`. The migration writes a new file and a `.migration.json` record (source SHA-256, tensors copied, tensors created and how, tensors dropped: always none, optimizer handling), never touches the source, copies every format-1 tensor unchanged, creates routers (normal 0.02) and experts (gate/up normal 0.02, down **zeros**, so the migrated model's outputs equal the original's), and carries AdamW moments over by parameter name.

Existing trained weights: none exist. No foundation model has been trained (GPU quota is 0); the only checkpoints ever produced were test and prototype runs. So nothing needed migrating, and the migration is verified on format-1 checkpoints written exactly as the old trainer wrote them. The tokenizer code and vocabulary layout are unchanged; no production tokenizer exists yet either.

Serving export: dense models export as Llama; models with experts export as Qwen2-MoE (router, routed experts with top-k and renormalized weights, shared FFN, dense-only layers). Logits match ours on CPU (test below).

Verification (tests/test_neural_core.py)

Claim	Test
Ten experts and routers are registered parameters, grouped by name	`test_ten_named_experts_and_registered_parameters`
Router keeps the top two and renormalizes; output equals the weighted sum of those experts	`test_router_picks_top_two_with_normalized_weights`
The forward pass calls the router and experts, and expert outputs change the logits	`test_forward_pass_calls_router_and_experts`
Router and selected experts receive gradients; unselected experts don't	`test_router_and_selected_experts_receive_gradients`
The optimizer holds exactly the trainable parameters (fails if any router/expert is left out)	`test_optimizer_holds_every_trainable_parameter`
Save and reload preserve config, every tensor and outputs; loading is strict	`test_save_reload_preserves_architecture_and_parameters`
Format-1 is refused, migrated to a new file, original unchanged, outputs equal, AdamW moments carried	`test_v1_checkpoint_is_refused_then_migrated_without_loss`

Claim	Test
A migrated checkpoint resumes training with experts	`test_migrated_checkpoint_resumes_training`
Training lowers loss, logs routing, resumes; held-out routing beats chance; ablation runs	`test_training_learns_records_routing_and_resumes`
Stopping and resuming gives the same weights as an uninterrupted run	`test_stop_and_resume_matches_an_uninterrupted_run`
Exported Qwen2-MoE gives the same logits as our model	`test_moe_export_matches_qwen2_moe`
Local inference streams through the experts (routing trace)	`test_inference_streams_through_the_experts`
The API server loads the configured checkpoint and streams tokens through the router	`test_api_server_loads_the_checkpoint_and_streams_model_tokens`
LoRA adapters never wrap experts or routers	`test_lora_adapters_leave_experts_and_router_alone`

Prototype results

Run on 2026-10-03 with `infra/local/run-prototype.sh` on 4 CPU cores. Raw results: `docs/results/`.

Setup. Tokenizer: byte-level BPE, 4,096 tokens, trained on the corpus below. Data (`proto-data-manifest.json`): ten domain files generated by `artemis/corpus.py` (Artemis AI original text, 3,000 documents each, label = domain) plus 80 CPython standard-library modules (PSF-2.0, unlabeled general text); 4.55M training tokens, 233K validation tokens, split by document. Every run: 2,000 steps × 16 × 256 tokens (8.2M tokens), AdamW, peak LR 1e-3, cosine schedule, stopped at step 1,000 and resumed from the checkpoint.

Run	Weights (total / per token)	Validation loss	Train time
`proto`: Prime Core + 10 experts, routing supervision 0.1	8.24M / 5.88M	0.977	1,499 s
`proto-unsup`: same, routing supervision off (control)	8.24M / 5.88M	0.974	1,474 s
`proto-dense`: dense baseline, matched weights per token	5.87M / 5.87M	**0.970**	**1,240 s**

The experts specialized (supervised run). On held-out validation text, the router put each domain's own expert in its top two for 99.6–99.9% of tokens at all three expert layers (chance: 20%). Removing one expert from routing raised its own domain's loss by 1.16–1.81

nats and changed the other nine domains by at most 0.001. The ablation criterion (own-domain increase is largest and more than twice the others) holds for all ten. At inference, a security prompt sent 40 of 40 generated tokens through the security expert at every expert layer; a cost prompt sent 40 of 40 through the cost expert.

Names alone did nothing (control). With routing supervision off, the experts' names meant nothing: the hit rate for each domain's namesake expert was 7–54% by layer (chance 20%), removing any expert changed any domain's loss by at most 0.03, and the expert whose removal hurt a domain most was usually a differently named one. The router still spread load and the model trained fine; the experts just didn't line up with their names.

Against the dense baseline: no gain at this scale. With the same weights per token and the same tokens, the dense model reached slightly lower validation loss (0.970 vs 0.977) and lower loss on every domain (by 0.001–0.012), and trained 17% faster on CPU (the expert dispatch is a Python loop). So this prototype shows that routing, specialization and ablation behave as designed. It does not show that experts make Artemis more capable or more efficient. That has to be measured at GPU scale on real data, where experts usually pay off through more total weights at the same cost per token.

Caveats: validation loss is dominated by the standard-library code (domain text loss is about 0.25); the domain text is template-generated and easy; specialization here is induced by labeled routing supervision, which needs labeled data at scale too.

What is not demonstrated

- General capability. The prototype corpus is generated template text; the model learns those patterns, not engineering skill.
- Any GPU run, any model above 8M parameters, vLLM serving of the MoE export, LoRA adapters on the MoE export in vLLM.
- Specialization on real, licensed domain data, which needs that data and GPU time.